



Συστήματα Ανάκτησης Πληροφοριών

Εργασία Φάση 2 2024-2025

Κεχριώτη Ελένη

3210078

Ισουφάι Ντέιβιντ

3210056

Εισαγωγή

Στην παρούσα εργασία υλοποιείται και αξιολογείται ένα σύστημα Ανάκτησης Πληροφορίας, με στόχο την αποτελεσματική αναζήτηση επιστημονικών εγγράφων. Το σύστημα βασίζεται στο **Elasticsearch**, μια ισχυρή μηχανή αναζήτησης ανοικτού κώδικα, και εφαρμόζεται πάνω στο σύνολο δεδομένων **SciDocs**, το οποίο περιλαμβάνει μεταδεδομένα, τίτλους και κείμενα επιστημονικών άρθρων. Η επικοινωνία με την Elastic Search γίνεται μέσω python client και της βιβλιοθήκης elasticsearch.

Στην παρούσα φάση επεκτείνουμε το σύστημα IR που αναπτύξαμε στη Φάση 2, αξιοποιώντας τεχνικές διανυσματικής αναπαράστασης λέξεων μέσω της χρήσης του αλγορίθμου Word2Vec. Στόχος της φάσης είναι η ενσωμάτωση σημασιολογικής εγγύτητας μεταξύ λέξεων, όχι μόνο με βάση τη λεξικογραφική συγγένεια, αλλά και με βάση τα συμφραζόμενα στα οποία αυτές εμφανίζονται μέσα στη συλλογή κειμένων. Για τον σκοπό αυτό, εκπαιδεύσαμε το δικό μας Word2Vec μοντέλο με τη χρήση της βιβλιοθήκης gensim, χρησιμοποιώντας ως είσοδο το πλήρες κείμενο των εγγράφων της συλλογής IR2025.

Για την εργασία θα πρέπει να κατεβάσετε τις βιβλιοθήκες elasticsearch και nltk.

```
> pip install nltk elasticsearch, gensim
```

Μεθοδολογία

Η μεθοδολογία της εργασίας ακολουθεί τα παρακάτω βασικά βήματα:

1. Bulk Indexing into Elastic Search

Κατά την φόρτωση των δεδομένων στο index, δημιουργούμε μια λίστα *sentences* η οποία αποθηκεύει τις προεπεξεργασμένες προτάσεις για την εκπαίδευση του *Word2Vec*. Συγκεκριμένα, αρχικά φορτώνουμε το json αρχείο (γραμμή - γραμμή) και δημιουργούμε ένα τελικό λεξικό (*final_doc*) στο οποίο προσθέτουμε όλες τις απαραίτητες πληροφορίες για το bulk indexing για κάθε καταχώρηση στο ευρετήριο. Έπειτα, συνενώνουμε όλα τα πεδία του ευρετηρίου και δημιουργούμε tokenized προτάσεις χρησιμοποιώντας την *simple_preprocess* η οποία:

- Κάνει tokenization και αφαίρεση σημείων στίξης και de-accent όπου είναι αναγκαίο.
- Μετατρέπει όλα τα γράμματα σε πεζά.
- Επιστρέφει λίστα όρων (tokens).

```
sentences = []
```

```

def build_bulk_doc(raw_line: str, fields):
    doc = json.loads(raw_line)
    meta = doc["metadata"]
    final_doc = {
        '_index': 'corpus_index',
        '_id': doc["_id"],
        '_source': {
            'title': doc['title'],
            'text': doc['text'],
            'authors': meta.get('authors', []),
            'year': meta.get('year'),
            'references': meta.get('references', []),
            'cited_by': meta.get('cited_by', 0)
        }
    }
    full_text = " ".join(str(doc[field]) for field in fields if
field in doc)
    tokens = simple_preprocess(full_text, deacc=True)
    if tokens:
        sentences.append(tokens)
    tokens = simple_preprocess(doc['text'], deacc=True)
    if tokens:
        sentences.append(tokens)

    return final_doc

with open("scidocs/corpus.jsonl", "r", encoding="utf-8") as file:
    bulk(
        client,
        (build_bulk_doc(line, fields = ["title", "text", "authors",
"year", "references", "cited_by"]) for line in file),
        refresh=True
    )

doc_count = client.count(index='corpus_index')['count']
print(f"Indexed {doc_count:,} documents in {'corpus_index'}")
print(f"Collected {len(sentences):,} tokenised sentences for
word2vec.")

```

2. Εκπαίδευση του μοντέλου Word2Vec

Σε αυτή την φάση δημιουργούμε ένα αντικείμενο Word2Vec από τη βιβλιοθήκη Gensim, το οποίο μαθαίνει τις διανυσματικές αναπαραστάσεις (*embeddings*) για κάθε λέξη. Οι παράμετροι που περνάμε στο αντικείμενο είναι οι εξής:

- **sentences=sentences:** παίρνει ως input τη λίστα με τις tokenized προτάσεις που συγκεντρώσαμε νωρίτερα.
- **vector_size=250:** μέγεθος κάθε διανύσματος λέξης.

- **window=4**: πόσες λέξεις πριν/μετά τη λέξη-στόχο λαμβάνονται υπόψη στο training.
- **min_count=4**: αγνοεί λέξεις που εμφανίζονται λιγότερο από 4 φορές συνολικά.
- **workers=4**: παράλληλοι «εργάτες» (νήματα) για γρηγορότερη εκπαίδευση.
- **epochs=10**: αρχικός αριθμός επαναλήψεων πάνω στο σύνολο δεδομένων.
- **sg=1**: χρησιμοποιεί το Skip-Gram (αντί του CBOW) αλγόριθμο

Έπειτα με την εντολή `build_vocab` κατασκευάζουμε το λεξιλόγιο του μοντέλου από τη λίστα προτάσεων και με την εντολή `train` βελτιστοποιούμε τα βάρη του (τα διανύσματα λέξεων) ώστε σημασιολογικά κοντινές λέξεις να βρίσκονται κοντά στον ενδιαμέσο χώρο.

```
model = Word2Vec(
    sentences=sentences,
    vector_size=250,
    window=4,
    min_count=4,
    workers= 4,
    epochs=10,
    sg=1 # 1 = skip-gram, 0 = CBOW
)
model.build_vocab(sentences)
model.train(sentences, total_examples=model.corpus_count, epochs=25)
```

3. Επεκταση Ερωτημάτων

Σε αυτό το βήμα επεκτείνουμε τα ερωτήματα τόσο με την χρήση Pos-tagging όσο και με την χρήση συνωνύμων από το Word2Vec.

Αρχικά δημιουργούμε μια λίστα με αγγλικά stop-words ώστε να αγνοήσουμε πολύ κοινούς όρους (and, or, is, at, etc.). Έπειτα σπάμε το query σε tokens, τα μετατρέπουμε σε μικρά και σε κάθε token εφαρμόζουμε μια ετικέτα που αντιστοιχεί στο μέρος του λόγου του (ουσιαστικό, ρήμα, κτλ.). Για κάθε ζεύγος (word, tag):

- αν η λέξη είναι στο **stop_words**, συνεχίζουμε στην επόμενη.
- βάζουμε πάντα στο **expanded** την ίδια τη λέξη του query.
- Αν η λέξη υπάρχει στο λεξιλόγιο του **model.wv** και έχουμε έγκυρο **WordNet POS**, τότε παίρνουμε τους topn πιο “παρόμοιους” γείτονες (με **most_similar**) και για κάθε (**neighbor, sim**), αν η ομοιότητα **sim** $\geq$ **sim_threshold**, τότε προσθέτουμε αυτόν τον γείτονα στο expanded.

Έτσι το expanded τελικά περιέχει: τις αρχικές λέξεις του query και τις συγγενικές τους λέξεις σύμφωνα με το Word2Vec.

```
from nltk import pos_tag
from nltk.corpus import stopwords
import string
```

```

# It turns POS tags from nltk(JJ, NN) in compatible format with
Wordent (wn.ADJ, wn.NOUN, etc)
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wn.ADJ
    elif treebank_tag.startswith('V'):
        return wn.VERB
    elif treebank_tag.startswith('N'):
        return wn.NOUN
    elif treebank_tag.startswith('R'):
        return wn.ADV
    else:
        return None

def expand_query_combined(query, model, sim_threshold=0.75, topn=3):
    stop_words = set(stopwords.words('english')) +
list(string.punctuation))
    tokens = word_tokenize(query.lower())
    tagged = pos_tag(tokens)

    expanded = set()

    for word, tag in tagged:
        if word in stop_words:
            continue
        wn_tag = get_wordnet_pos(tag)
        expanded.add(word)

        if word in model.wv and wn_tag is not None:
            for neighbor, sim in model.wv.most_similar(word,
topn=topn):
                if sim >= sim_threshold:
                    expanded.add(neighbor)

    return " ".join(expanded)

```

4. Hyperparameter Tuning

Για να καταφέρουμε να λάβουμε τα καλύτερα δυνατά αποτελέσματα δημιουργήσαμε ένα τεχνητό pipeline το οποίο εφαρμόζει την λογική του gridsearch, ώστε να βρούμε τις καλύτερες παραμέτρους για το αντικείμενο του Word2Vec. Επειδή όμως λόγω περιορισμένων υπολογιστικών πόρων δεν καταφέραμε να το υποστηρίξουμε και αναγκαστήκαμε να διακόψουμε την διαδικασία πρόωρα. Έτσι τελικά τον ακόλουθο κώδικα δεν το συμπεριλάβαμε στο notebook, αλλά τον συμπεριλαμβάνουμε εδώ ως στάδιο στην ανάπτυξη του τελικού κώδικα.

```

from time import time
from sklearn.model_selection import ParameterGrid

```

```

queries_file = "scidocs/queries.jsonl"
run_file      = "trec_eval/my_results"
index_name    = "corpus_index"
top_k         = [20]
run_tag       = "my_run"

#expanding queries with similarity threshold and top n most similar
def expand_query(query):
    tokens = [t for t in simple_preprocess(query) if t not in
stopwords]
    expanded = set(tokens)
    for tok in tokens:
        if tok not in model.wv: continue
        for neigh, sim in model.wv.most_similar(tok, topn=topn):
            if sim < sim_threshold: break
            expanded.add(neigh)
    return " ".join(expanded)

#loading queries
queries = {}
with open(queries_file, "r", encoding="utf-8") as f:
    for line in f:
        o = json.loads(line)
        queries[o["_id"]] = o["text"]

# load qrels once
qrels_path = "trec_eval/test_trec.tsv"

# hyperparameter grid
param_grid = {
    "vector_size": [100, 200, 250],
    "window":      [3, 4, 5],
    "min_count":   [1, 2, 5],
    "sg":          [0, 1],
    "negative":    [5, 10],
    "epochs":      [5, 10],
}

best_score, best_cfg = -1, None

for cfg in ParameterGrid(param_grid):
    print(f"→ Training W2V with {cfg} ...", end=" ")
    t0 = time()
    # train
    m = Word2Vec(
        sentences=sentences,
        vector_size=cfg["vector_size"],
        window=cfg["window"],
        min_count=cfg["min_count"],
        sg=cfg["sg"],
        negative=cfg["negative"],

```

```

        workers=4,
        epochs=cfg["epochs"],
        seed=42
    )
    model = m

    # label configuration for filenames
    label = (

f"vs{cfg['vector_size']}_w{cfg['window']}_mc{cfg['min_count']}"
        f"_sg{cfg['sg']}_neg{cfg['negative']}_e{cfg['epochs']}"
    )

    # MAP for each top_k (we just kept k=20 for computational
    reasons so this is not much needed and the avg_map is going to be
    equal to the top_20 map
    sum_map = 0.0
    for k in top_k:
        run_path = f"{run_file}_{label}_{k}.run"
        # write the run file
        with open(queries_file, "r", encoding="utf-8") as qf, \
            open(run_path, "w", encoding="utf-8") as rf:

            for line in qf:
                obj = json.loads(line)
                qid, text = obj["_id"], obj["text"]
                exp = expand_query(text)

                resp = client.search(
                    index=index_name,
                    size=k,
                    query={
                        "bool": {
                            "should": [
                                {"match": {"title": {"query":
text, "boost": 3}}},
                                {"match": {"text": {"query":
text, "boost": 3}}},
                                {"match": {"allContent": {"query":
text, "boost": 3}}},
                                {"match": {"title": {"query":
exp, "boost": 1}}},
                                {"match": {"text": {"query":
exp, "boost": 1}}},
                                {"match": {"allContent": {"query":
exp, "boost": 1}}}
                            ]
                        }
                    }
                )

                for rank, hit in enumerate(resp["hits"]["hits"],
start=1):
                    rf.write(

```

```

f"{qid} Q0 {hit['_id']} {rank}
{hit['_score']:.4f} {run_tag}\n"
)

# run trec_eval and parse MAP
result = subprocess.run(
    ["/trec_eval/trec_eval", "-m", "map", qrels_path,
run_path],
    capture_output=True, text=True
)
map_score = float(result.stdout.strip().split()[-1])
print(f"top{k} MAP={map_score:.4f}", end=" ")
sum_map += map_score

avg_map = sum_map / len(top_k)
elapsed = time() - t0
print(f"→ avg MAP={avg_map:.4f} ({elapsed:.0f}s)")

if avg_map > best_score:
    best_score, best_cfg = avg_map, cfg

print(f"\n Best avg MAP={best_score:.4f} with config: {best_cfg}")

```

Μέρος των αποτελεσμάτων φαίνεται στην παρακάτω εικόνα:

```

→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 200, 'window': 3} ... top20 MAP=0.1033 → avg MAP=0.1033 (75s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 200, 'window': 4} ... top20 MAP=0.1047 → avg MAP=0.1047 (79s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 200, 'window': 5} ... top20 MAP=0.1038 → avg MAP=0.1038 (75s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 250, 'window': 3} ... top20 MAP=0.1032 → avg MAP=0.1032 (90s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 250, 'window': 4} ... top20 MAP=0.1044 → avg MAP=0.1044 (88s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 0, 'vector_size': 250, 'window': 5} ... top20 MAP=0.1042 → avg MAP=0.1042 (90s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 100, 'window': 3} ... top20 MAP=0.1029 → avg MAP=0.1029 (78s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 100, 'window': 4} ... top20 MAP=0.1020 → avg MAP=0.1020 (92s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 100, 'window': 5} ... top20 MAP=0.1027 → avg MAP=0.1027 (107s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 200, 'window': 3} ... top20 MAP=0.1046 → avg MAP=0.1046 (115s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 200, 'window': 4} ... top20 MAP=0.1038 → avg MAP=0.1038 (128s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 200, 'window': 5} ... top20 MAP=0.1036 → avg MAP=0.1036 (146s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 250, 'window': 3} ... top20 MAP=0.1046 → avg MAP=0.1046 (141s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 250, 'window': 4} ... top20 MAP=0.1044 → avg MAP=0.1044 (159s)
→ Training W2V with {'epochs': 5, 'min_count': 1, 'negative': 10, 'sg': 1, 'vector_size': 250, 'window': 5} ... top20 MAP=0.1039 → avg MAP=0.1039 (180s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 100, 'window': 3} ... top20 MAP=0.1043 → avg MAP=0.1043 (36s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 100, 'window': 4} ... top20 MAP=0.1039 → avg MAP=0.1039 (36s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 100, 'window': 5} ... top20 MAP=0.1040 → avg MAP=0.1040 (36s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 200, 'window': 3} ... top20 MAP=0.1036 → avg MAP=0.1036 (47s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 200, 'window': 4} ... top20 MAP=0.1038 → avg MAP=0.1038 (48s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 200, 'window': 5} ... top20 MAP=0.1035 → avg MAP=0.1035 (48s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 250, 'window': 3} ... top20 MAP=0.1034 → avg MAP=0.1034 (56s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 250, 'window': 4} ... top20 MAP=0.1037 → avg MAP=0.1037 (61s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 0, 'vector_size': 250, 'window': 5} ... top20 MAP=0.1031 → avg MAP=0.1031 (60s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 1, 'vector_size': 100, 'window': 3} ... top20 MAP=0.1032 → avg MAP=0.1032 (51s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 1, 'vector_size': 100, 'window': 4} ... top20 MAP=0.1036 → avg MAP=0.1036 (57s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 1, 'vector_size': 100, 'window': 5} ... top20 MAP=0.1031 → avg MAP=0.1031 (66s)
→ Training W2V with {'epochs': 5, 'min_count': 2, 'negative': 5, 'sg': 1, 'vector_size': 200, 'window': 3} ... top20 MAP=0.1045 → avg MAP=0.1045 (67s)

```


Αποτελέσματα

Τα αποτελέσματα για κάθε φάση για τα τοπ 20,30,50 του MAP καθώς και η μέση ακρίβεια για τα πρώτα 5,10,15,20 φαίνεται παρακάτω.

Η αξιολόγηση της σάσης 3 δείχνει μικρή αλλά σταθερή βελτίωση σε σχέση με τις φάσεις 1 και 2. Συγκεκριμένα:

- Στο MAP η φάση 3 πέτυχε το υψηλότερο score με μέγιστο το 0.1101 ξεπερνώντας τη Φάση 2 (max 0.1066) και τη Φάση 1 (max 0.1078). Αυτό δείχνει ότι η ενσωμάτωση σημασιολογικής πληροφορίας από τα συμφραζόμενα των λέξεων βοηθά στην ανάκτηση πιο σχετικών εγγράφων.
- Σε όλες τις μετρικές Precision (P@k) παρατηρείται σταδιακή βελτίωση, κάτι που επιβεβαιώνει ότι η μέθοδος επέκτασης των queries συνέβαλε θετικά στην ακρίβεια των πρώτων αποτελεσμάτων.

Φάση 3	20	30	50
map	0.1047	0.1076	0.1101
avgPre@5	0.1198	0.1198	0.1198
avgPre@10	0.0860	0.0860	0.0860
avgPre@15	0.0665	0.0665	0.0665
avgPre@20	0.0558	0.0558	0.0558

Φάση 3	20	30	50
recall_5	0.1214	0.1214	0.1214
recall_10	0.1744	0.1744	0.1722
recall_15	0.2022	0.2022	0.2022
recall_20	0.2262	0.2262	0.2262
recall_30	0.2262	0.2584	0.2584
recall_100	0.2262	0.2584	0.3002
recall_200	0.2262	0.2584	0.3002

recall_500	0.2262	0.2584	0.3002
recall_1000	0.2262	0.2584	0.3002

Φάση 2	20	30	50
map	0.1010	0.1040	0.1066
avgPre@5	0.1162	0.1162	0.1162
avgPre@10	0.0814	0.0814	0.0814
avgPre@15	0.0647	0.0647	0.0647
avgPre@20	0.0542	0.0542	0.0542

Φάση 1	20	30	50
map	0.1019	0.1051	0.1078
avgPre@5	0.1160	0.1160	0.1160
avgPre@10	0.0826	0.0826	0.0826
avgPre@15	0.0649	0.0649	0.0649
avgPre@20	0.0541	0.0541	0.0541

Παρότι οι διαφορές είναι σχετικά μικρές, η Φάση 3 αναδείχθηκε ως η πιο αποτελεσματική. Αυτό καταδεικνύει ότι ο συνδυασμός λεξικογραφικών και σημασιολογικών τεχνικών (WordNet + Word2Vec) μπορεί να ενισχύσει την απόδοση ενός συστήματος IR, ειδικά όταν αυτό λειτουργεί πάνω σε επιστημονικά κείμενα όπου η σημασία των όρων είναι κρίσιμη. Τα word embeddings χαρτογραφούν καλύτερα την σημασιολογική απόσταση μεταξύ των λέξεων σε σύγκριση με τις άλλες δύο μεθόδους, με αποτέλεσμα τα σχετικά κείμενα να ανακτώνται γρηγορότερα και υψηλότερα (μεγαλύτερο ranking) με αποτέλεσμα και το Mean Average Precision να αυξάνεται. Τέλος, η χρήση του Word2Vec, μας δίνει την δυνατότητα να μάθουμε και να προσαρμοστούμε στις “ιδιαιτερότητες” του τρέχοντος corpus, καταφέροντας έτσι να έχουμε corpus-specific-knowledge, σε αντίθεση με το WordNet το οποίο περιοριζόταν σε προκαθορισμένα synsets.